



# SMART CONTRACT AUDIT REPORT

for

ParaSwap AugustusFees



Prepared By: Xiaomi Huang

PeckShield  
June 6, 2024

## Document Properties

|                |                             |
|----------------|-----------------------------|
| Client         | ParaSwap                    |
| Title          | Smart Contract Audit Report |
| Target         | AugustusFees                |
| Version        | 1.0.1                       |
| Author         | Xuxian Jiang                |
| Auditors       | Jason Shen, Xuxian Jiang    |
| Reviewed by    | Xiaomi Huang                |
| Approved by    | Xuxian Jiang                |
| Classification | Public                      |

## Version Info

| Version | Date         | Author(s)    | Description          |
|---------|--------------|--------------|----------------------|
| 1.0.1   | June 6, 2024 | Xuxian Jiang | Post Release #1      |
| 1.0     | May 29, 2024 | Xuxian Jiang | Final Release        |
| 1.0-rc1 | May 23, 2024 | Xuxian Jiang | Release Candidate #1 |

## Contact

For more information about this document and its contents, please contact PeckShield Inc.

|       |                        |
|-------|------------------------|
| Name  | Xiaomi Huang           |
| Phone | +86 183 5897 7782      |
| Email | contact@peckshield.com |

Contents

1 Introduction 4

1.1 About ParaSwap . . . . . 4

1.2 About PeckShield . . . . . 5

1.3 Methodology . . . . . 5

1.4 Disclaimer . . . . . 7

2 Findings 9

2.1 Summary . . . . . 9

2.2 Key Findings . . . . . 10

3 Detailed Results 11

3.1 Simplified Surplus Handling Without Partners . . . . . 11

4 Conclusion 13

References 14



# 1 | Introduction

Given the opportunity to review the design document and related source code of the `AugustusFees` smart contract, we outline in the report our systematic approach to evaluate potential security issues in the smart contract implementation, expose possible semantic inconsistencies between smart contract code and design document, and provide additional suggestions or recommendations for improvement. Our results show that the given version of smart contracts is well-engineered without security concerns. This document outlines our audit results.

## 1.1 About ParaSwap

`ParaSwap` is a decentralized finance (DeFi) platform that aggregates liquidity from various decentralized exchanges (DEXs) to facilitate swift and efficient token swaps. It optimizes trades for the best available rates, minimizes slippage, and provides users with a user-friendly interface for seamless cryptocurrency trading across multiple blockchain networks. This audit covers the latest version of `AugustusFees` for trading fee handling and sharing with partner network. The basic information of `AugustusFees` is as follows:

Table 1.1: Basic Information of `AugustusFees`

| Item                | Description                                             |
|---------------------|---------------------------------------------------------|
| Issuer              | ParaSwap                                                |
| Website             | <a href="https://paraswap.io/">https://paraswap.io/</a> |
| Type                | Ethereum Smart Contract                                 |
| Platform            | Solidity                                                |
| Audit Method        | Whitebox                                                |
| Latest Audit Report | June 6, 2024                                            |

In the following, we show the Git repository of reviewed files and the commit hash value used in this audit. Note this audit covers only the `AugustusFees` smart contract.

- <https://github.com/paraswap/paraswap-augustus.git> (22a9ba09)

And here is the commit ID after all fixes for the issues found in the audit have been checked in:

- <https://github.com/paraswap/paraswap-augustus.git> (f000850)

## 1.2 About PeckShield

PeckShield Inc. [5] is a leading blockchain security company with the goal of elevating the security, privacy, and usability of current blockchain ecosystems by offering top-notch, industry-leading services and products (including the service of smart contract auditing). We are reachable at Telegram (<https://t.me/peckshield>), Twitter (<http://twitter.com/peckshield>), or Email ([contact@peckshield.com](mailto:contact@peckshield.com)).

Table 1.2: Vulnerability Severity Classification

| Impact     | High     | Medium | Low    |
|------------|----------|--------|--------|
|            | Critical | High   | Medium |
|            | High     | Medium | Low    |
| Low        | Medium   | Low    | Low    |
| Likelihood |          |        |        |
|            |          |        |        |
|            |          |        |        |

## 1.3 Methodology

To standardize the evaluation, we define the following terminology based on the OWASP Risk Rating Methodology [4]:

- Likelihood represents how likely a particular vulnerability is to be uncovered and exploited in the wild;
- Impact measures the technical loss and business damage of a successful attack;
- Severity demonstrates the overall criticality of the risk.

Likelihood and impact are categorized into three ratings: *H*, *M* and *L*, i.e., *high*, *medium* and *low* respectively. Severity is determined by likelihood and impact and can be classified into four categories accordingly, i.e., *Critical*, *High*, *Medium*, *Low* shown in Table 1.2.

To evaluate the risk, we go through a checklist of items and each would be labeled with a severity category. For one check item, if our tool or analysis does not identify any issue, the contract

Table 1.3: The Full Audit Checklist

| Category                    | Checklist Items                           |
|-----------------------------|-------------------------------------------|
| Basic Coding Bugs           | Constructor Mismatch                      |
|                             | Ownership Takeover                        |
|                             | Redundant Fallback Function               |
|                             | Overflows & Underflows                    |
|                             | Reentrancy                                |
|                             | Money-Giving Bug                          |
|                             | Blackhole                                 |
|                             | Unauthorized Self-Destruct                |
|                             | Revert DoS                                |
|                             | Unchecked External Call                   |
|                             | Gasless Send                              |
|                             | Send Instead Of Transfer                  |
|                             | Costly Loop                               |
|                             | (Unsafe) Use Of Untrusted Libraries       |
|                             | (Unsafe) Use Of Predictable Variables     |
|                             | Transaction Ordering Dependence           |
|                             | Deprecated Uses                           |
| Semantic Consistency Checks | Semantic Consistency Checks               |
| Advanced DeFi Scrutiny      | Business Logics Review                    |
|                             | Functionality Checks                      |
|                             | Authentication Management                 |
|                             | Access Control & Authorization            |
|                             | Oracle Security                           |
|                             | Digital Asset Escrow                      |
|                             | Kill-Switch Mechanism                     |
|                             | Operation Trails & Event Generation       |
|                             | ERC20 Idiosyncrasies Handling             |
|                             | Frontend-Contract Integration             |
|                             | Deployment Consistency                    |
|                             | Holistic Risk Management                  |
| Additional Recommendations  | Avoiding Use of Variadic Byte Array       |
|                             | Using Fixed Compiler Version              |
|                             | Making Visibility Level Explicit          |
|                             | Making Type Inference Explicit            |
|                             | Adhering To Function Declaration Strictly |
|                             | Following Other Best Practices            |

is considered safe regarding the check item. For any discovered issue, we might further deploy contracts on our private testnet and run tests to confirm the findings. If necessary, we would additionally build a PoC to demonstrate the possibility of exploitation. The concrete list of check items is shown in Table 1.3.

In particular, we perform the audit according to the following procedure:

- Basic Coding Bugs: We first statically analyze given smart contracts with our proprietary static code analyzer for known coding bugs, and then manually verify (reject or confirm) all the issues found by our tool.
- Semantic Consistency Checks: We then manually check the logic of implemented smart contracts and compare with the description in the white paper.
- Advanced DeFi Scrutiny: We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.
- Additional Recommendations: We also provide additional suggestions regarding the coding and development of smart contracts from the perspective of proven programming practices.

To better describe each issue we identified, we categorize the findings with Common Weakness Enumeration (CWE-699) [3], which is a community-developed list of software weakness types to better delineate and organize weaknesses around concepts frequently encountered in software development. Though some categories used in CWE-699 may not be relevant in smart contracts, we use the CWE categories in Table 1.4 to classify our findings. Moreover, in case there is an issue that may affect an active protocol that has been deployed, the public version of this report may omit such issue, but will be amended with full details right after the affected protocol is upgraded with respective fixes.

## 1.4 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of the given smart contract(s) or blockchain software, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, we always recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contract(s). Last but not least, this security audit should not be used as investment advice.

Table 1.4: Common Weakness Enumeration (CWE) Classifications Used in This Audit

| Category                                             | Summary                                                                                                                                                                                                                                                                                                     |
|------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Configuration</b>                                 | Weaknesses in this category are typically introduced during the configuration of the software.                                                                                                                                                                                                              |
| <b>Data Processing Issues</b>                        | Weaknesses in this category are typically found in functionality that processes data.                                                                                                                                                                                                                       |
| <b>Numeric Errors</b>                                | Weaknesses in this category are related to improper calculation or conversion of numbers.                                                                                                                                                                                                                   |
| <b>Security Features</b>                             | Weaknesses in this category are concerned with topics like authentication, access control, confidentiality, cryptography, and privilege management. (Software security is not security software.)                                                                                                           |
| <b>Time and State</b>                                | Weaknesses in this category are related to the improper management of time and state in an environment that supports simultaneous or near-simultaneous computation by multiple systems, processes, or threads.                                                                                              |
| <b>Error Conditions, Return Values, Status Codes</b> | Weaknesses in this category include weaknesses that occur if a function does not generate the correct return/status code, or if the application does not handle all possible return/status codes that could be generated by a function.                                                                     |
| <b>Resource Management</b>                           | Weaknesses in this category are related to improper management of system resources.                                                                                                                                                                                                                         |
| <b>Behavioral Issues</b>                             | Weaknesses in this category are related to unexpected behaviors from code that an application uses.                                                                                                                                                                                                         |
| <b>Business Logic</b>                                | Weaknesses in this category identify some of the underlying problems that commonly allow attackers to manipulate the business logic of an application. Errors in business logic can be devastating to an entire application.                                                                                |
| <b>Initialization and Cleanup</b>                    | Weaknesses in this category occur in behaviors that are used for initialization and breakdown.                                                                                                                                                                                                              |
| <b>Arguments and Parameters</b>                      | Weaknesses in this category are related to improper use of arguments or parameters within function calls.                                                                                                                                                                                                   |
| <b>Expression Issues</b>                             | Weaknesses in this category are related to incorrectly written expressions within code.                                                                                                                                                                                                                     |
| <b>Coding Practices</b>                              | Weaknesses in this category are related to coding practices that are deemed unsafe and increase the chances that an exploitable vulnerability will be present in the application. They may not directly introduce a vulnerability, but indicate the product has not been carefully developed or maintained. |



## 2 | Findings

### 2.1 Summary

Here is a summary of our findings after analyzing the implementation of the `AugustusFees` protocol. During the first phase of our audit, we study the smart contract source code and run our in-house static code analyzer through the codebase. The purpose here is to statically identify known coding bugs, and then manually verify (reject or confirm) issues reported by our tool. We further manually review business logic, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.

| Severity      | # of Findings |                                                                                     |
|---------------|---------------|-------------------------------------------------------------------------------------|
| Critical      | 0             |                                                                                     |
| High          | 0             |                                                                                     |
| Medium        | 0             |                                                                                     |
| Low           | 0             |                                                                                     |
| Informational | 1             |  |
| Total         | 1             |                                                                                     |

We have so far identified a list of potential issues: some of them involve subtle corner cases that might not be previously thought of, while others refer to unusual interactions among multiple contracts. For each uncovered issue, we have therefore developed test cases for reasoning, reproduction, and/or verification. After further analysis and internal discussion, we determined a few issues of varying severities need to be brought up and paid more attention to, which are categorized in the above table. More information can be found in the next subsection, and the detailed discussions of each of them are in [Section 3](#).

## 2.2 Key Findings

Overall, these smart contracts are well-designed and engineered, though the implementation can be improved by resolving the identified issues (shown in Table 2.1), including 1 informational recommendation.

Table 2.1: Key AugustusFees Audit Findings

| ID      | Severity      | Title                                                        | Category         | Status   |
|---------|---------------|--------------------------------------------------------------|------------------|----------|
| PVE-001 | Informational | <a href="#">Simplified Surplus Handling Without Partners</a> | Coding Practices | Resolved |

Beside the identified issues, we emphasize that for any user-facing applications and services, it is always important to develop necessary risk-control mechanisms and make contingency plans, which may need to be exercised before the mainnet deployment. The risk-control mechanisms should kick in at the very moment when the contracts are being deployed on mainnet. Please refer to Section 3 for details.



## 3 | Detailed Results

### 3.1 Simplified Surplus Handling Without Partners

- ID: PVE-001
- Severity: Informational
- Likelihood: N/A
- Impact: N/A
- Target: AugustusFees
- Category: Coding Practices [2]
- CWE subcategory: CWE-1041 [1]

#### Description

The Augustus protocol aggregates liquidity from various DEXs to facilitate swift and efficient token swaps with improved user experience. It features a new fee model contract AugustusFees. In the process of understanding and analyzing the new fee model, we notice a possible (minor) improvement for code simplification.

In the following, we show the code snippet from the related `processSwapExactAmountInFeesAndTransfer()` routine. The code snippet is designed to handle surplus and return the rest to beneficiary. Specifically, when the partner address is not specified (as `0x0`), if slippage is positive, the protocol will take the surplus and transfer the rest to the beneficiary. Similarly, if there is no positive slippage, transfer the received amount to the beneficiary. We notice possible redundancy in the following two cases: `blacklistedTokens[destToken]` (line 199) and `surplus==0` (line 209). With that, we can simply consolidate them.

```
196     if (surplus > 0) {
197         // If the token is blacklisted, send the received amount to the beneficiary
198         // we won't process fees
199         if (blacklistedTokens[destToken]) {
200             // transfer the received amount to the beneficiary, keeping 1 wei dust
201             _transferAndLeaveDust(destToken, beneficiary, receivedAmount);
202             return (receivedAmount - 1, 0, 0);
203         }
204         // transfer the remaining amount to the beneficiary, keeping 1 wei dust
205         _transferAndLeaveDust(destToken, beneficiary, remainingAmount);
206         // transfer the surplus to the fee wallet
```

```
207         destToken.safeTransfer(feeWallet, surplus);
208         return (remainingAmount - 1, surplus, 0);
209     } else {
210         // transfer the received amount to the beneficiary, keeping 1 wei dust
211         _transferAndLeaveDust(destToken, beneficiary, receivedAmount);
212         return (receivedAmount - 1, 0, 0);
213     }
```

Listing 3.1: Original Code Snippet From `AugustusFees::processSwapExactAmountInFeesAndTransfer()`

**Recommendation** Revisit the above-mentioned routines for improved code simplicity. An example revision is shown below. Note the same improvement can be applied to another routine, i.e., `processSwapExactAmountInFeesAndTransferUniV3()`, from the same contract.

```
196         if (surplus == 0 || blacklistedTokens[destToken]) {
197             // transfer the received amount to the beneficiary, keeping 1 wei dust
198             _transferAndLeaveDust(destToken, beneficiary, receivedAmount);
199             return (receivedAmount - 1, 0, 0);
200         } else {
201             // transfer the remaining amount to the beneficiary, keeping 1 wei dust
202             _transferAndLeaveDust(destToken, beneficiary, remainingAmount);
203             // transfer the surplus to the fee wallet
204             destToken.safeTransfer(feeWallet, surplus);
205             return (remainingAmount - 1, surplus, 0);
206         }
```

Listing 3.2: Revised Code Snippet From `AugustusFees::processSwapExactAmountInFeesAndTransfer()`

**Status** This issue has been resolved as the team aims to minimize the gas cost when there is no surplus. For the suggested revision, it will make an additional `SLOAD` when there is no surplus.

## 4 | Conclusion

In this audit, we have analyzed the `AugustusFees` design and implementation. By gathering liquidity from the main decentralized exchanges together, `ParaSwap` is a middleware streamlining user's interactions with various `DeFi` services. This audit covers the latest version of `AugustusFees` for trading fee handling and sharing with partner network. The current code base is well structured and neatly organized. Those identified issues are promptly confirmed and fixed.

Meanwhile, we need to emphasize that `Solidity`-based smart contracts as a whole are still in an early, but exciting stage of development. To improve this report, we greatly appreciate any constructive feedbacks or suggestions, on our methodology, audit findings, or potential gaps in scope/coverage.



## References

- [1] MITRE. CWE-1041: Use of Redundant Code. <https://cwe.mitre.org/data/definitions/1041.html>.
- [2] MITRE. CWE CATEGORY: Bad Coding Practices. <https://cwe.mitre.org/data/definitions/1006.html>.
- [3] MITRE. CWE VIEW: Development Concepts. <https://cwe.mitre.org/data/definitions/699.html>.
- [4] OWASP. Risk Rating Methodology. [https://www.owasp.org/index.php/OWASP\\_Risk\\_Rating\\_Methodology](https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology).
- [5] PeckShield. PeckShield Inc. <https://www.peckshield.com>.